Q-MODE: Amino Acid Lattice Mapping for Binding-Pocket Representation and Quantum-Assisted Docking Site Search

Author Name(s)
Multidisciplinary Project

July 27, 2026

Abstract

Q-MODE is a computational pipeline that converts a protein binding pocket, supplied as a PDB structure, into a discrete, spatially-consistent sequence of pharmacophoric interaction sites, and subsequently encodes that sequence into qubit-ready quantum states. The pipeline combines classical cheminformatics (RDKit-based feature extraction, Crippen and Abraham solvation-derived intensities, solvent-accessibility filtering, covalent-topology-based ordering) with a quantum-inspired encoding and search stage, following the two-step scheme (first/second encoding) and the modified Grover-search formulation introduced in *"Quantum algorithm for protein-ligand docking sites identification in the interaction space"*. This report documents the full functioning of the pipeline as currently implemented: the residue- and ligand-parsing stage, the pharmacophore and intensity-assignment stage, the surface and deduplication filters, the topological chain assembly, the quantum encodings, and the Grover-search-based candidate ranking. The validation and benchmarking section is intentionally left incomplete in this revision and will be completed in a subsequent iteration of the report.

Contents

1 Introduction

Protein-ligand docking site identification is a central problem in structure-based drug design: given a protein binding pocket, one wants to identify which sub-regions of that pocket are chemically compatible with a candidate ligand's pharmacophoric profile. Classical docking pipelines typically rely on scoring functions evaluated over a continuous 3D search space, which is expensive to sample exhaustively.

Q-MODE explores an alternative representation: it reduces the binding pocket to a *flat, ordered sequence of pharmacophoric sites* — a "1D chain" analogous in spirit to a residue sequence — and then encodes short, ligand-sized windows of that chain into low-qubit-count quantum states. This representation is directly compatible with the amplitude- and basis-state-encoding schemes used by quantum search algorithms (in particular, a modified Grover search), enabling a quantum-inspired (and, in the encoding stage, genuinely quantum-circuit-executed) approach to scanning candidate docking windows.

1.1 Motivation

Two properties of a binding pocket motivate the lattice/chain representation used here:

- **Locality**: a ligand interacts with a contiguous, spatially compact set of pocket residues. Representing the pocket as an ordered sequence lets a "sliding window" of fixed size stand in for a candidate docking sub-site.
- **Discretization**: encoding each site's chemical character (hydrophobic vs. polar/charged) as a small number of qubits per site keeps the resulting quantum circuits within the qubit counts realistically simulable today (or executable on near-term hardware), while still preserving the essential pharmacophoric signal needed to discriminate candidate sites.

1.2 Scientific Background

The quantum-encoding and quantum-search stages of Q-MODE follow the scheme proposed in *"Quantum algorithm for protein-ligand docking sites identification in the interaction space"* (Liliopoulos et al.):

- A **first encoding** that binarizes, per site, a hydrophobicity channel (h) and a hydrogen-bonding/charge channel (hb) into a 2-qubit basis state, so that a window of k sites maps onto a $2k$-qubit computational basis state. This is the representation used to build the Grover oracle.
- A **second encoding** that maps the same two channels onto two pairs of probability amplitudes (a, b) and (c, d), suitable for amplitude-based (e.g. SWAP-test) similarity/distance estimation between two sites or windows.
- A **modified Grover search**, run per "protein shift" (i.e. per tiling offset of the pocket chain into non-overlapping ligand-sized windows), that identifies which windows' first-encoding bitstring matches the ligand's own bitstring with probability above the uniform ($1/N$) threshold.

Q-MODE implements the first encoding, the second encoding, and the modified Grover search (oracle and diffusion operator, executed on a Qiskit simulator backend) as described above. The paper's own SWAP-test-based ranking of candidate sites is **not yet implemented**; Q-MODE currently substitutes a classical Euclidean-distance-to-ligand-centroid ranking for benchmark validation, as detailed in Section 3.11.

2 Pipeline Overview

Given a pocket PDB file (and, optionally, a full PDB structure containing a bound ligand), Q-MODE executes the following stages in sequence:

1. **Residue extraction** — parse residues and real 3D atomic coordinates from the PDB file; assign bond orders by structural template comparison rather than positional overlay (Section 3.1).
2. **Pharmacophoric feature computation** — RDKit-based atom/group feature extraction (Section 3.2).
3. **Intensity assignment** — Crippen LogP contributions for hydrophobic sites; Abraham solute descriptors for H-bond donor/acceptor/ionizable sites (Section 3.3).
4. **Surface filtering** — SASA-based removal of solvent-inaccessible (buried) sites (Section 3.4).
5. **Site deduplication** — merging of near-duplicate same-type sites into a single centroid site (Section 3.5).
6. **Topological ordering** — within each residue, ordering sites by a breadth-first traversal of the covalent bond graph, starting from the backbone nitrogen (Section 3.6).
7. **Chain assembly** — concatenation of residues in protein-chain order into one flat sequence (Section 3.7).
8. **Quantum encoding** — first (binary) and second (amplitude) encodings of ligand-sized sliding-window segments (Section 3.8).
9. **Ligand extraction** *(optional)* — parsing of a bound ligand's HETATM group and construction of its own pharmacophore chain (Section 3.9).
10. **Grover search** *(optional)* — quantum-circuit-based identification of candidate docking windows matching the ligand's encoded profile (Section 3.10).
11. **Distance-to-ligand validation** *(optional, benchmark mode)* — see Section 3.11, left to be completed.

Figure 1 sketches the data flow between stages.

`PDB file` → `ResidueRecord[]` → `AtomFeature[]` (per residue) → intensity assignment → surface filter → dedup → topological order → **flat_chain** → quantum encoding (first/second) → **quantum_chain**

(optional) `ligand PDB` → ligand chain → Grover search against **flat_chain** → **grover_search** candidates → distance-to-ligand validation

Figure 1: High-level data flow of the Q-MODE pipeline.

3 Methods

3.1 Residue Extraction and Bond-Order Assignment

Each residue is read directly from the PDB's `ATOM`/`HETATM` records (`pdb_reader.py`), keeping only the primary alternate location (`altLoc` blank or `A`) and, by default, discarding water. Residues

are grouped by (`chain_id`, `res_seq`, `res_name`) and sorted by (`chain_id`, `res_seq`) to preserve chain order.

For each residue, an RDKit molecule is built from the real heavy-atom coordinates via a PDB block, and bond orders/aromaticity are assigned by *template comparison* (`AssignBondOrdersFromTemplate`) against a canonical SMILES template for each of the 20 standard amino acids (plus histidine protonation variants `HSD/HSE/HSP/HIE/HID/HIP` and the disulfide-bonded `CYX`). Two templates are used per residue type: a "free amino acid" template (full `-COOH`) when a terminal `OXT` atom is present, and an "internal" template (backbone `-OH` removed) otherwise — since most residues in a chain are peptide-bonded and not free termini. Missing hydrogens (not resolved by X-ray) are added afterward with 3D coordinates consistent with the heavy-atom geometry (`AddHs(addCoords=True)`).

This template-based approach is deliberately preferred over a purely geometric/positional bond-order guess: it is robust to the moderate coordinate noise present in crystallographic structures, at the cost of requiring the residue type to be one of the recognized standard (or near-standard) amino acids.

3.2 Pharmacophoric Feature Extraction

Each residue's RDKit molecule is passed through RDKit's chemical feature factory (`BaseFeatures.fdef`), which identifies six pharmacophore families, mapped onto the internal type vocabulary used throughout Q-MODE:

RDKit family	**Q-MODE type**
Donor	`HBondDonor`
Acceptor	`HBondAcceptor`
Hydrophobe / LumpedHydrophobe	`Hydrophobe`
Aromatic	`Aromatic`
PosIonizable	`PosIonizable`
NegIonizable	`NegIonizable`

Each feature carries the 3D centroid of its constituent atom(s) and, for multi-atom groups, an atom-index list used by later stages (intensity re-assignment, surface filtering, topological mapping). Where the feature factory is unavailable, a manual fallback identifies donors/acceptors from N/O/F valence and hydrogen count, and hydrophobic sites from per-atom Crippen contributions directly.

3.3 Intensity Assignment

Every site carries a scalar *intensity*, intended to reflect the physical strength of that site's interaction potential rather than a placeholder constant.

Hydrophobic sites. `Hydrophobe` sites are assigned the sum of positive per-atom Crippen LogP contributions (`rdMolDescriptors._CalcCrippenContribs`) over their constituent atoms, falling back to a neutral value of 1.0 when no positive contribution is present.

H-bond donor/acceptor/ionizable sites. These sites are assigned an *intrinsic*, per-site hydrogen-bonding intensity from Abraham's Linear Solvation Energy Relationship (LSER) scales — specifically α_2^H (donor strength) for `HBondDonor`/`PosIonizable` sites and β_2^H (acceptor strength) for `HBondAcceptor`/`NegIonizable` sites (`abraham_hbond.py`). Values are looked up in a table keyed by (`res_name`, `pdb_atom_name`) (with a wildcard residue name for backbone atoms shared by all amino acids), cross-checked against the UFZ-LSER database. When the exact

residue/atom key is not present — most notably for ligand atoms, which never match a standard-amino-acid key — a heuristic functional-group classifier (`_heuristic_group_key`) inspects the atom's local chemical environment (element, valence, neighboring atoms) and reuses the closest protein-side proxy already in the table (carboxyl, amine, phenol, thiol); anything outside those four groups keeps the neutral default intensity.

Two known approximations are carried through explicitly rather than silently:

- Abraham's scales are defined for *neutral* solutes. Ionized side chains (protonated Lys/Arg, deprotonated Asp/Glu) are proxied by their nearest neutral analog (e.g. propylamine for the lysine ammonium group), which is expected to *underestimate* the true interaction strength of the charged form. Each affected table entry is flagged with an explicit `note` field.
- The ligand's own H-bond intensity has no direct Abraham entry (the table is indexed by amino-acid residue/atom names), so it is covered only by the heuristic group-key fallback described above, and otherwise remains at the neutral default.

3.4 Surface Filtering

Buried sites are not physically reachable by an incoming ligand and would otherwise add noise to the pocket representation. Q-MODE filters sites by per-atom Solvent-Accessible Surface Area (SASA), computed with BioPython's Shrake–Rupley implementation (probe radius 1.4 Å) over the *whole* structure (`surface_filter.py`). A site is kept if at least one of its constituent atoms has SASA $> \tau$, with default threshold $\tau = 1.0$ Å^2 (a conservative value; the literature range for "significantly exposed" spans roughly 1.0–5.0 Å^2). This filter is on by default and can be disabled with `-no-surface-filter`, e.g. to retain buried sites for structural comparison rather than ligandability analysis.

Because RDKit atom indices and PDB atom-name bookkeeping can become misaligned during template-based bond assignment, the matching between a feature and its SASA value is done by *nearest 3D coordinate* to a known PDB atom rather than by index, which is more robust to that misalignment.

3.5 Site Deduplication (Centroid Merging)

Within a residue, several nearby atoms of the same pharmacophore type (e.g. multiple ring carbons contributing separate `Aromatic` features) would otherwise appear as separate, highly-correlated sites. `site_dedup_centroid.py` greedily groups same-type features whose pairwise Euclidean distance is below 1.5 Å, replacing each group with a single site at the group's coordinate centroid and an aggregate intensity equal to the *sum* of the group's individual intensities. Same-type sites that are spatially far apart (e.g. two separate H-bond acceptors on opposite sides of a residue) are correctly kept as distinct sites. The result is additionally capped to the `max_sites` most intense sites per residue (default 12 for protein residues; configurable and set lower, e.g. 3, for the ligand side to keep the Grover-search qubit count tractable).

3.6 Topological Ordering

Within a residue, the deduplicated sites are ordered by a breadth-first search (BFS) over the residue's covalent bond graph, starting from the (lowest-index) backbone nitrogen atom (`site_selection.py`). Each site is first mapped to its nearest heavy atom by 3D distance, after which the BFS visit order over the bond graph determines the site's position in the residue-local sequence. This produces a chemically meaningful, deterministic ordering (ties broken by sorted neighbor traversal) that reflects the residue's actual connectivity rather than an arbitrary or purely spatial order.

3.7 Chain Assembly

Residues are concatenated in protein-chain order — i.e. by (`chain_id`, `res_seq`) as already established at the parsing stage — and each residue's internally-ordered sites are appended in sequence. The result is one flat list, `flat_chain`, where every entry carries: the originating residue label, residue name/sequence/chain id, 3D coordinates, pharmacophore type, and (channel-appropriate) intensity. This flat chain is the shared representation consumed by both the visualization utilities and the quantum-encoding stage.

3.8 Quantum Encoding

Each site in `flat_chain` is reduced to two scalar channels (`qubit_chain.get_h_hb_intensities`):

$$(h, hb) = \begin{cases} (\text{intensity}, 0) & \text{type} \in \{\texttt{Hydrophobe}, \texttt{Aromatic}\} \\ (0, \text{intensity}) & \text{type} \in \{\texttt{HBondDonor}, \texttt{HBondAcceptor}, \texttt{PosIonizable}, \texttt{NegIonizable}\} \end{cases}$$

First encoding (basis-state binarization). Each channel is binarized against a per-channel threshold, producing a 2-bit basis state per site:

$$q_h = \begin{cases} 0 & h < h_{\text{thr}} \\ 1 & h \geq h_{\text{thr}} \end{cases} \qquad q_{hb} = \begin{cases} 0 & hb < hb_{\text{thr}} \\ 1 & hb \geq hb_{\text{thr}} \end{cases}$$

The paper's reference scheme thresholds at the channel's range midpoint $(v_{\min} + v_{\max})/2$. Empirically, real pocket intensity distributions are right-skewed (most sites sit near the floor, with a few strong outliers), which drags the mean upward and leaves almost nothing above threshold — collapsing the chain to an all-zero bitstring. Q-MODE instead thresholds at the *median* of each channel's active (nonzero) values, which splits the population roughly 50/50 and keeps the binarization informative for the downstream Grover search (`compute_h_hb_thresholds`).

A window of k consecutive sites is thus mapped onto a $2k$-bit basis state $|x\rangle = \left| q_h^{(1)} q_{hb}^{(1)} \ldots q_h^{(k)} q_{hb}^{(k)} \right\rangle$.

Second encoding (amplitude encoding). Each channel value v is also mapped onto a pair of normalized probability amplitudes via a cosine/sine-like projection of v within its channel's range $[v_{\min}, v_{\max}]$:

$$p = v_{\max} - v, \qquad q = v - v_{\min}, \qquad (a, b) = \left(\frac{p}{\sqrt{p^2 + q^2}}, \frac{q}{\sqrt{p^2 + q^2}} \right)$$

applied independently to h (giving a, b with $a^2 + b^2 = 1$) and to hb (giving c, d with $c^2 + d^2 = 1$), with v clamped to $[v_{\min}, v_{\max}]$ beforehand so that a site outside its own channel's range (e.g. $h = 0$ for a purely polar site) lands at the range's lower extreme instead of producing an invalid (unnormalizable) state. This representation is intended for amplitude-based (e.g. SWAP-test) similarity/distance estimation between sites, as prescribed by the reference paper.

Sliding-window segmentation. `qubit_chain.build_qubit_chain` slides a window of size `ligand_size` (default 3 sites) over `flat_chain` with unit stride, producing one `QubitSegment` per window position, each carrying its first-encoding bitstring and its per-site second-encoding amplitudes. This is the "classical" quantum-encoding step (Step 7 in the pipeline enumeration), independent of the real ligand used later by Grover search.

3.9 Ligand Extraction

When a full PDB structure containing a bound ligand is supplied (`-ligand-pdb`), `ligand_reader.py` parses its `HETATM` records, excluding water and the 20 standard amino acids, and groups atoms by (`chain_id`, `res_seq`, `res_name`) — i.e. by physical copy, since a small/symmetric structure's asymmetric unit may contain several bound copies of the same ligand. The largest non-water, non-amino-acid group (by heavy-atom count) is auto-selected by default, or a specific one via `-ligand-code`.

Bond orders for the ligand are assigned, in order of preference, by:

1. Fetching the ideal SMILES for the ligand's 3-letter code from the RCSB PDB Chemical Component Dictionary (CCD) REST API and applying the same template-based assignment used for amino acids (`AssignBondOrdersFromTemplate`).
2. Falling back to geometric bond-order perception (`rdDetermineBonds`) when the CCD entry is unavailable (unknown code, or no network access).

The extracted ligand is then passed through the same local pipeline as a protein residue — feature extraction, Abraham/heuristic intensity assignment, centroid deduplication (capped to `-ligand-max-sites`, default 3), and topological ordering — yielding a ligand-side chain of the same shape as a protein window, suitable for direct comparison in the Grover search stage.

3.10 Grover Search

For a ligand chain of k sites, Q-MODE runs a modified Grover search (`qmode/grover/search.py`) against every non-overlapping tiling of the protein's `flat_chain` into windows of size k:

1. **Tiling.** For each shift offset $o \in \{0, \ldots, k-1\}$, the pocket chain is partitioned into non-overlapping windows of size k starting at o (`tile_offset`). Each window is reduced to its first-encoding bitstring; when several windows collapse onto the same bitstring, the one with the highest aggregate h/hb intensity across its sites (the "most interactive" window) is retained as that bitstring's representative.
2. **Superposition state.** The unique bitstrings observed at this offset are combined into an equal superposition,
$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{i=1}^{N} |x_i\rangle ,$$
over $n_{\text{qubits}} = 2k$ qubits, where N is the number of unique windows at this offset.
3. **Oracle.** The ligand's own first-encoding bitstring x_{lig} defines a phase-flip oracle,
$$\hat{O} = I - 2\,|x_{\text{lig}}\rangle\,\langle x_{\text{lig}}| .$$
4. **Diffusion operator.** The Grover diffusion operator about the superposition state,
$$\hat{G} = 2\,|s\rangle\,\langle s| - I,$$
is applied after the oracle.
5. **Circuit execution.** $|s\rangle$ is prepared with Qiskit's `StatePreparation`, followed by the oracle and diffusion operators (each compiled from their dense unitary matrix via `UnitaryGate`), then measured. The circuit is transpiled and executed on Qiskit Aer's `AerSimulator` (default 4096 shots), yielding an empirical probability for each measured bitstring.

6. **Acceptance threshold.** A shift offset's search is considered a match if the ligand bitstring's measured probability exceeds the uniform baseline $1/N$ — i.e. the oracle+diffusion step measurably amplified that state above what a uniform superposition alone would give.

Across all shift offsets, matching windows are collected into a candidate list, each carrying: the shift offset, the window's start index in `flat_chain`, its *interactivity score* (aggregate h/hb intensity), the residues it spans, the matched bitstring, its measured probability, the $1/N$ threshold, and the number of unique states at that offset. Candidates are sorted by descending interactivity score, so that the most chemically interactive matching window is reported first.

Scalability. Oracle and diffusion operators are synthesized from dense $2^{n_{\text{qubits}}} \times 2^{n_{\text{qubits}}}$ unitary matrices. Qiskit's generic unitary-synthesis routine does not scale gracefully past roughly 10 qubits (tens of seconds to minutes per shift offset in practice), which is why `-ligand-max-sites` defaults to 3 (6 qubits); raising it substantially increases circuit-compilation time.

3.11 Distance-to-Ligand Validation

Validation and benchmarking section — to be completed. This includes: the classical Euclidean-distance-to-ligand-centroid stand-in currently implemented in `qmode/grover/evaluate.py` (as an interim substitute for the reference paper's SWAP-test-based ranking); quantitative results over the expanded benchmark set (Astex Diverse Set targets plus the four hand-picked complexes in `make_pockets.py`); accuracy/success-rate metrics; and a discussion of where the current ranking succeeds or fails relative to the true bound pose.

4 Implementation

4.1 Repository Structure

```
Q-MODE-project/
|-- qmode/
|   |-- pdb_reader.py           # PDB parsing -> residues, real 3D coords
|   |-- ligand_reader.py        # Ligand HETATM parsing (CCD template / geometric
    fallback)
|   |-- feature_extraction.py   # RDKit pharmacophore features + Crippen h
|   |-- abraham_hbond.py        # Abraham hb intensity lookup
|   |-- surface_filter.py       # SASA-based solvent-exposure filter
|   |-- site_selection.py       # Topological (BFS) site ordering
|   |-- site_dedup_centroid.py  # Near-duplicate site merging
|   |-- quantum_encoding.py     # First/second quantum encoding
|   |-- qubit_chain.py          # Sliding-window segmentation + qubit chain
|   |-- grover/
|   |   |-- search.py           # Oracle, diffusion, circuit execution
|   |   `-- evaluate.py         # Distance-to-ligand validation
|   |-- lattice_fitting.py      # 3D->2D PCA projection (utility, unused by main
    pipeline)
|   |-- snapping.py             # 2D->integer lattice nodes (utility, unused)
|   `-- labeling.py             # One-hot pharmacophore labeling (utility, unused)
|-- scripts/
|   |-- run_pipeline.py         # Main CLI entry point
|   |-- plot_residue_chain.py   # Single-residue 3D + 1D visualization
|   `-- make_pockets.py         # Downloads sample PDBs, crops binding pockets
|-- data/raw/                   # Input / generated pocket PDB files
`-- tests/                      # Unit tests
```

4.2 Dependencies

Package	Minimum version
rdkit	2023.3.1
numpy	1.24
pandas	2.0
scikit-learn	1.3
scipy	1.11
matplotlib	3.7
biopython	1.79
qiskit	2.2
qiskit-aer	0.17
tqdm	4.65

4.3 Command-Line Interface

The pipeline is exposed through a single script, `scripts/run_pipeline.py`:

```
python scripts/run_pipeline.py --pdb data/raw/3PTB_pocket.pdb --plot

python scripts/run_pipeline.py --pdb data/raw/3PTB_pocket.pdb \
    --output data/processed/ --save-plot data/processed/pocket.png

python scripts/run_pipeline.py --pdb data/raw/4dfr_pocket.pdb \
    --ligand-pdb data/raw/4DFR.pdb
```

Option	Default	Description
`-pdb`	required	Path to the pocket (or whole-protein) PDB file.
`-output`	`None`	Directory for JSON/CSV output files.
`-plot`	`False`	Show an interactive visualization.
`-save-plot`	`None`	Save plots as PNG images.
`-no-surface-filter`	filter on	Disable the SASA solvent-exposure filter.
`-sasa-threshold`	`1.0`	SASA threshold ($Å^2$) for solvent exposure.
`-ligand-size`	`3`	Sliding-window size (sites) for the classical quantum-chain segmentation (Section 3.8).
`-ligand-pdb`	`None`	Path to a PDB file with the ligand's HETATM records; enables Grover search.
`-ligand-code`	`None`	3-letter ligand code to disambiguate multiple HETATM groups.
`-ligand-max-sites`	`3`	Max ligand pharmacophore sites used by Grover search (qubit-count control).

4.4 Output Artifacts

Running the pipeline with `-output` produces up to three JSON/CSV artifacts:

`pocket_chain.json` / `.csv` The full flat lattice chain with per-site metadata (residue label, coordinates, pharmacophore type, intensity).

`quantum_chain.json` Sliding-window segments with their first-encoding bitstring and second-encoding amplitudes.

`grover_search.json` *(only when `-ligand-pdb` is given)* Grover candidates ranked by descending interactivity score, each including its distance-to-ligand validation field once Section 3.11 is completed.

4.5 Testing

Unit tests (`tests/`) cover the ligand reader, the Grover search circuit construction, and an end-to-end pipeline smoke test:

```
pytest tests/
```

5 Known Limitations

- **Abraham proxies for ionized groups.** Abraham's LSER scales are defined for neutral solutes; ionized side chains (protonated Lys/Arg, deprotonated Asp/Glu) are approximated by their nearest neutral analog, which likely underestimates true interaction strength.
- **Ligand H-bond intensity coverage.** The Abraham table is indexed by amino-acid residue/atom name and does not natively cover ligand atoms; only four functional-group classes (carboxyl, amine, phenol, thiol) are covered by the heuristic fallback, with everything else defaulting to a neutral intensity.
- **Qubit-count scalability.** Qiskit's generic unitary synthesis for the Grover oracle/diffusion operators does not scale past roughly 10 qubits, bounding `-ligand-max-sites` to small values (default 3, i.e. 6 qubits) for practical runtimes.
- **SWAP-test ranking not implemented.** The reference paper's own quantum (amplitude/SWAP-test-based) ranking of candidate docking sites is not implemented; Q-MODE substitutes a classical Euclidean-distance-to-ligand-centroid ranking, valid only in benchmark mode where the true ligand pose is known (see Section 3.11, to be completed).
- **Utility modules not wired into the main pipeline.** `lattice_fitting.py`, `snapping.py`, and `labeling.py` implement a 3D→2D PCA projection, integer-lattice snapping, and one-hot pharmacophore labeling respectively, but are not currently called from `run_pipeline.py`.

6 Conclusion and Future Work

Q-MODE implements a complete, end-to-end path from a raw PDB binding pocket to a quantum-circuit-executed Grover search over candidate docking windows, grounded in physically-derived (Crippen/Abraham) site intensities rather than placeholder values. The classical feature-extraction and chain-assembly stages (Sections 3.1–3.7) are stable and unit-tested; the quantum-encoding and Grover-search stages (Sections 3.8–3.10) faithfully reproduce the reference paper's first encoding, second encoding, oracle, and diffusion operator.

The immediate next step for this project is completing Section 3.11: a quantitative validation of the Grover-search candidate ranking against the expanded benchmark set (the Astex Diverse Set targets plus the originally hand-picked complexes), reporting distance-to-ligand accuracy and discussing whether — and how — a genuine SWAP-test-based amplitude ranking would change the results relative to the current classical distance stand-in.

References

1. Liliopoulos et al., *Quantum algorithm for protein-ligand docking sites identification in the interaction space.*

2. Abraham, M. H. *Scales of solute hydrogen-bonding: their construction and application to physicochemical and biochemical processes*, Chem. Soc. Rev. (1993/1994) and subsequent updates (2000–2012); values cross-checked against the UFZ-LSER Database (https://www.ufz.de/lserd).

3. Wildman, S. A.; Crippen, G. M. *Prediction of Physicochemical Parameters by Atomic Contributions*, J. Chem. Inf. Comput. Sci. (1999).

4. Shrake, A.; Rupley, J. A. *Environment and exposure to solvent of protein atoms. Lysozyme and insulin*, J. Mol. Biol. (1973).

5. Hartshorn, M. J. et al. *Diverse, High-Quality Test Set for the Validation of Protein-Ligand Docking Performance*, J. Med. Chem. (2007) — source of the Astex Diverse Set benchmark targets.